

Lab 3: Architectural Justification

Problem Statement #38: Dropshipping Inventory & Order Orchestrator

Architecture Selection

Microservices Architecture was chosen for the Dropshipping Inventory & Order Orchestrator System. The system is decomposed into seven components: Webhook Gateway, Message Broker, Order Orchestrator, Inventory Service, Supplier Integration Service, Tracking Service and Notification Service, plus an Order Database. They communicate only through the provided/required interfaces shown in the component diagram.

Style	Strength for this system	Weakness for this system
Layered	Simple, clear separation of concerns.	One deployable unit: a slow supplier call or a webhook spike affects everything; cannot scale ingestion alone.
Client-Server	Central control, easy data consistency.	Single server is a bottleneck and single point of failure for webhooks and supplier calls.
Microservices (chosen)	Independent scaling, fault isolation, async messaging, per-service deployment.	Operational complexity and eventual consistency (accepted; see below).

Two Reasons for the Choice

- Fault isolation and independent scaling. Supplier APIs are third-party, slow and occasionally down, while storefront webhooks arrive in bursts (flash sales). As separate services, a failing supplier API only affects the Supplier Integration and Inventory services; the Webhook Gateway keeps accepting orders, and each service can be scaled on its own.
- Supplier variety and change. Each supplier has a different API. All supplier-specific logic sits behind the Supplier Integration Service interfaces (IStockLookup, IPurchaseOrder), so the Order Orchestrator's multi-vendor order splitting (FR-003) does not change when a supplier is added or its API changes, and each service can be redeployed on its own.

Security Advantage

The Webhook Gateway is the single, hardened entry point (NFR-002): it enforces HTTPS, verifies the storefront's HMAC signature and rate-limits requests before anything reaches the internal services. Supplier API credentials are held only by the Supplier Integration Service, so a compromise of another service does not expose them. Internal calls are authenticated with service tokens or mutual TLS.

Performance Benefit

The Webhook Gateway only validates the request and publishes it to the Message Broker (IOrderQueue), then returns HTTP 200 immediately, meeting NFR-001 (acknowledgement within 200 ms). Slow work (supplier stock checks, purchase orders) runs asynchronously, which prevents webhook retry storms; consumers scale horizontally with queue depth.

Accepted Trade-off

Microservices add operational complexity and eventual consistency. We accept this and mitigate it with idempotency keys on the storefront order ID, so retried or duplicate webhooks never create duplicate supplier purchase orders.